



TourGuide

Documentation fonctionnelle et technique

Sommaire

1. Présentation du projet

1.1 Objectifs du projet

1.2 Hors du champ d'application

1.3 Mesures du projet

2. Spécifications fonctionnelles

3. Spécifications techniques

3.1 Schémas de conception technique

3.2 Glossaire

3.3 Solutions techniques

3.4 Autres solutions non retenues

1. Présentation du projet

Le projet TourGuide est une application Java permettant de suivre la position de nombreux utilisateurs et de leur attribuer des récompenses lorsqu'ils visitent des attractions proches.

Le public cible est composé de services touristiques et d'agences souhaitant fidéliser leurs clients via des récompenses automatiques basées sur la géolocalisation.

L'avantage commercial réside dans l'automatisation et la scalabilité du système : le service reste performant même avec un grand nombre d'utilisateurs simultanés.

1.1 Objectifs du projet

Mettre en place une application capable de suivre en temps réel la localisation des utilisateurs.

Attribuer automatiquement des récompenses selon leur proximité avec des attractions touristiques.

Garantir des performances même avec un volume élevé d'utilisateurs (jusqu'à 100 000).

1.2 Hors du champ d'application

L'interface graphique n'est pas développée dans ce projet (back-end uniquement).

Le déploiement sur mobile n'est pas prévu.

La gestion avancée des profils utilisateurs (préférences complexes, sécurité OAuth2) n'est pas incluse.

1.3 Mesures du projet

Succès fonctionnel : passage de tous les tests unitaires et d'intégration.

Succès technique : temps d'exécution des tests de performance en dessous des seuils définis (par ex. < 15 minutes pour 100 000 utilisateurs).

Succès qualité : pipeline CI opérationnel sur GitHub Actions, garantissant compilation + tests à chaque push.

2. Spécifications fonctionnelles

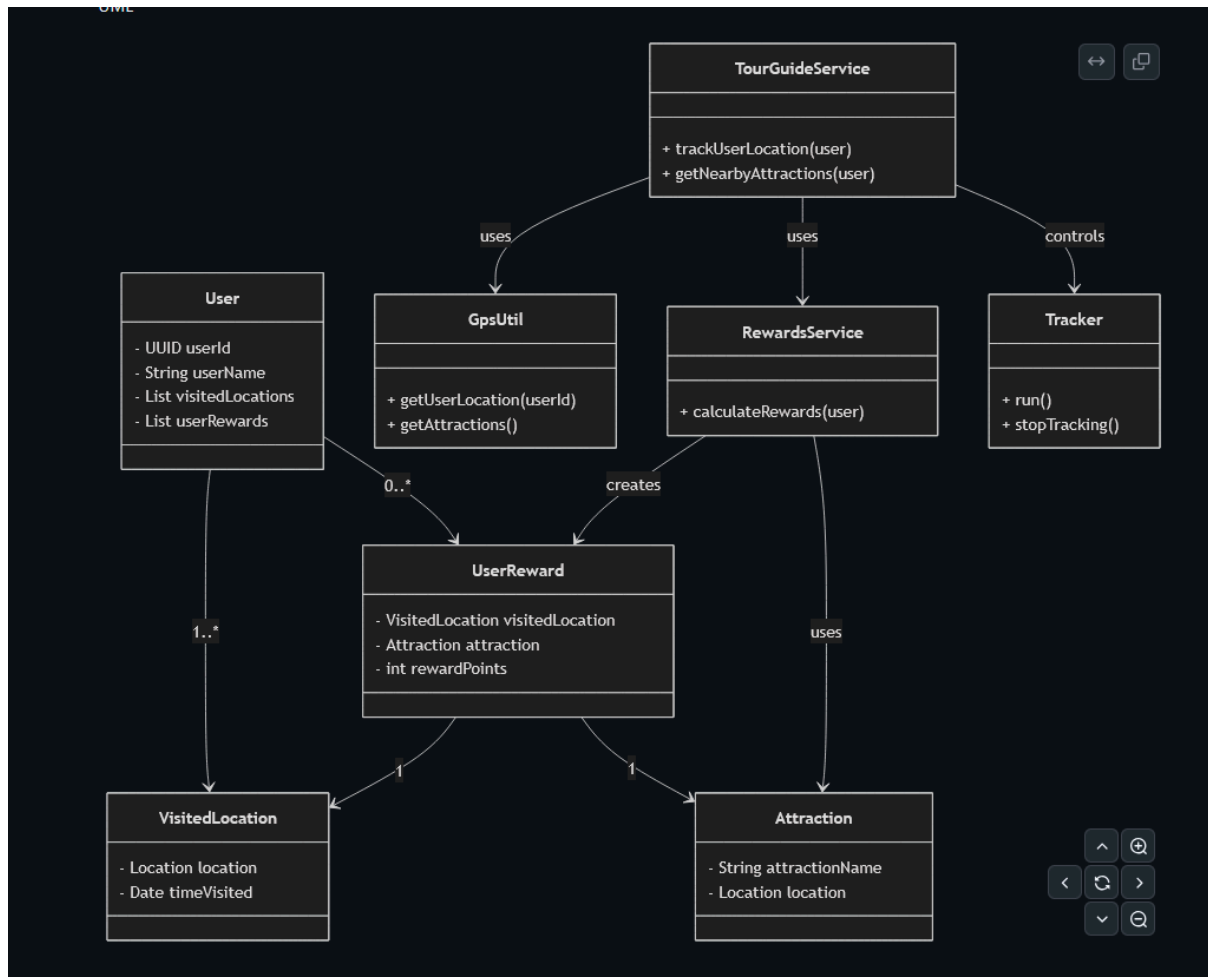
Fonctionnalités implémentées :

- Suivi de la localisation des utilisateurs.
- Attribution de récompenses en fonction des attractions proches.
- Gestion d'un volume élevé d'utilisateurs.
- Tests de performance et optimisation via multi-threading (CompletableFuture).
- Pipeline CI/CD avec GitHub Actions.

3. Spécifications techniques

3.1 Schémas de conception technique

Diagramme UML



- Service GpsUtil → récupération des positions.
- Service RewardsService → calcul des récompenses.
- Service TourGuideService → orchestre le tout.
- Tracker → exécute le suivi en tâche de fond
- Utilisation de threads et CompletableFuture pour paralléliser le traitement.

3.2 Glossaire

Attraction : lieu touristique.

Reward : points attribués à un utilisateur.

VisitedLocation : position enregistrée.

CI/CD : intégration continue.

3.3 Solutions techniques

- Langage : Java 17
- Gestion des dépendances : Maven
- Concurrency : CompletableFuture + ForkJoinPool
- Sécurité thread : CopyOnWriteArrayList pour visitedLocations et userRewards
- CI/CD : GitHub Actions avec build/test/package automatisés
- Tests : JUnit 5, tests unitaires + performance

3.4 Autres solutions non retenues

- Synchronisation classique (synchronized) → moins efficace.
- Jenkins ou GitLab CI : écartés au profit de GitHub Actions (simplicité, intégration directe).